

Programación de Sistemas

Temas 1.1 a 2.3

Práctica 1

Del programa fuente a los tokens: analizador léxico para IMP

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2
Colegio de Ciencia y Tecnología, UACM

Esta práctica acompaña la transición desde el panorama de la programación de sistemas hasta la construcción del primer componente del compilador. Avanzaremos, de manera guiada, hacia las expresiones regulares, los autómatas finitos y la implementación de un analizador léxico independiente con PYTHON y LARK.

Alcance

Construiremos y probaremos únicamente el **analizador léxico**. La práctica no decidirá todavía si una secuencia de tokens forma una expresión o un comando válido; esa tarea corresponde al análisis sintáctico. Tampoco asignará tipos ni manejará alcances.

Propósito y producto

Al terminar, podrás:

- explicar por qué el analizador léxico es una fase del compilador y cuál es su contrato;
- derivar categorías léxicas a partir de una descripción del lenguaje fuente;
- expresar patrones mediante expresiones regulares y relacionarlos con autómatas finitos;
- implementar con LARK un lexer independiente para la convención concreta de IMP;
- conservar atributos y una tabla inicial de localidades;
- probar entradas correctas, conflictos entre categorías y caracteres no reconocidos.

Producto esperado

Una carpeta `practical1_analizador_lexico` que contenga los scripts completados, `requirements.txt`, ejemplos, pruebas, un `README.md` y el reporte solicitado para el portafolio. El lexer deberá mostrar por cada token su tipo, lexema, línea y columna.

Ruta guiada

Paso	Trabajo principal	Producto parcial
0	Preparar el entorno y ejecutar la versión mínima.	Entorno reproducible.
1	Recuperar el pipeline y el contrato léxico.	Mapa entrada-salida.
2	Leer la definición de IMP y fijar su escritura concreta.	Inventario de tokens.
3	Pasar de patrones a expresiones regulares y AFD.	AFD de localidades.
4	Diseñar reglas, atributos y casos de prueba.	Tabla léxica aprobada.
5	Extender el lexer mínimo con LARK.	Secuencia de tokens.
6	Registrar localidades y comprobar posiciones.	Tabla inicial.
7	Probar errores y preparar la entrega.	Evidencias y reporte.

Forma de trabajo

En cada paso: predice el resultado, realiza un cambio pequeño, ejecuta una prueba, compara con otra persona y registra la evidencia. Si el resultado no coincide, explica la diferencia antes de continuar.

Paso 0. Preparar el entorno

0.1 Copiar el punto de partida

Copia la carpeta `scripts_estudiante` a tu repositorio personal y renómbrala como `practica1_analizador_lexico`. Conserva los archivos y los `TODO`: indican el orden de trabajo.

Estructura inicial:

```
practica1_analizador_lexico/  
+-- README.md  
+-- requirements.txt  
+-- afd_localidades.py  
+-- lexer_imp.py  
+-- probar_lexer.py  
+-- pruebas_lexer.py  
+-- ejemplos/  
    +-- minimo.imp  
    +-- flujo.imp  
    +-- control.imp  
    +-- error.imp
```

0.2 Crear un entorno reproducible

Ejecuta los comandos de `README.md`. Comprueba que `requirements.txt` registra una versión concreta de LARK. No instales dependencias adicionales para resolver tareas que pueden realizarse con PYTHON, LARK y `unittest`.

0.3 Ejecutar antes de modificar

```
python probar_lexer.py ejemplos/minimo.imp  
python -m unittest -v pruebas_lexer.py
```

Registra:

- salida observada: _____
- pruebas ejecutadas: _____
- pruebas omitidas y motivo: _____

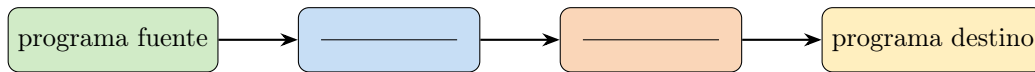
Checkpoint 0

☐ La entrada X produce un token LOC. ☐ Las pruebas habilitadas terminan correctamente. ☐
El primer commit conserva el punto de partida.

Paso 1. Del software de sistemas al contrato léxico

1.1 Recuperar el recorrido

Completa el esquema con las representaciones involucradas en la transformación.



1.2 Formular el contrato

Completa antes de escribir código:

Entrada	_____
Transformación	Agrupar _____, clasificarlos y conservar _____.
Salida	_____
Consumidor	_____

Ejemplo de una transformación, todavía sin reglas oficiales

Para el texto $X := 25$, una salida descriptiva podría separar X , $:=$ y 25 . La pregunta léxica es qué caracteres forman cada unidad y qué categoría comunica cada una. La pregunta de si la asignación está bien construida se atenderá después.

Checkpoint 1. Límite de responsabilidad

Explica por qué encontrar X no permite decidir si fue declarada ni qué valor tiene:

Paso 2. Conocer el lenguaje fuente

2.1 Sintaxis abstracta de IMP

Winskel presenta IMP mediante números N , valores de verdad T , localidades Loc , expresiones aritméticas $Aexp$, expresiones booleanas $Bexp$ y comandos Com . Sus reglas de formación pueden resumirse así:

$$\begin{aligned} a &::= n \mid X \mid a_0 + a_1 \mid a_0 - a_1 \mid a_0 \times a_1, \\ b &::= true \mid false \mid a_0 = a_1 \mid a_0 \leq a_1 \mid \neg b \mid b_0 \wedge b_1 \mid b_0 \vee b_1, \\ c &::= skip \mid X := a \mid c_0; c_1 \mid if\ b\ then\ c_0\ else\ c_1 \mid while\ b\ do\ c. \end{aligned}$$

Estas reglas indican qué construcciones existen, pero no fijan por sí solas todos los detalles de su escritura concreta ni las precedencias.

Convención concreta del curso

Para archivos de texto usaremos símbolos ASCII. Esta tabla es una decisión de representación del compilador del curso, no una ampliación de la sintaxis abstracta.

Winskel	Archivo fuente	Decisión
$\times$	$*$	multiplicación
$\leq$	$<=$	comparación menor o igual
$\neg$	$!$	negación
$\wedge$	$\&\&$	conjunción
$\vee$	$ $	disyunción
$n \in N$	NUM sin signo	el signo menos se emite como token independiente
agrupación concreta	(y)	delimitadores para la gramática posterior

2.2 Inventario de tokens

Completa la columna de atributo. Escribe «no requiere» cuando el nombre del token comunica toda la información necesaria.

Grupo	Tokens y escritura	Atributo
Datos	LOC: letras seguidas opcionalmente por dígitos	
Datos	NUM: uno o más dígitos	
Valores	TRUE: true; FALSE: false	
Comandos	SKIP, IF, THEN, ELSE, WHILE, DO	
Aritmética	PLUS, MINUS, TIMES	
Booleanos	EQ, LE, NOT, AND, OR	
Control	ASSIGN, SEMI, LPAR, RPAR	
Omitido	espacios, tabuladores y saltos de línea	

Checkpoint 2

1. ¿Por qué `if` no debe emitirse como LOC?
2. ¿Por qué esta práctica tokeniza - por separado?
3. ¿Qué parte de la tabla deberá respetar la práctica sintáctica?

Paso 3. De patrones a autómatas

3.1 De una descripción a una expresión regular

Una expresión regular permite especificar con precisión un conjunto de lexemas. Usaremos concatenación, alternativa, clausura $*$, repetición positiva $+$, opción $?$ y clases de caracteres.

Modelo: numerales

«uno o más dígitos» $\longrightarrow$ $[0-9]^+$

Cadena	¿Satisface?	Razón
0	sí	contiene un dígito
25	sí	contiene dos dígitos
cadena vacía	no	se exige al menos uno
-3	no como un solo NUM	- será otro token

3.2 Derivar el patrón de localidades

La descripción de trabajo es: «una o más letras seguidas opcionalmente por cero o más dígitos».

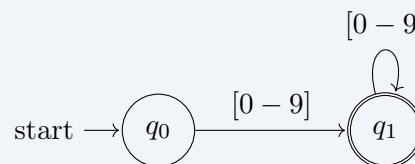
LOC = _____

Prueba tu propuesta con:

Cadena	Predicción	Justificación
X		
contador		
X2		
2X		
X_2		
X2Y		

Modelo: AFD de uno o más dígitos

El estado recuerda si todavía no se ha leído ningún dígito o si ya se leyó al menos uno.



3.3 Convertir y completar el AFD de localidades

Para $[A-Za-z]^+[0-9]^*$, un AFN puede usar una transición ϵ para pasar de la zona de letras a la zona de dígitos. Al construir el AFD, cada estado representa un conjunto de estados alcanzables del AFN. Después se eliminan estados inalcanzables y se comparan estados para decidir si son

distinguibles.

Completa la tabla del AFD resultante. Usa L para letra, D para dígito y O para otro carácter.

Estado	L	D	O	¿final?
q_0	q_{letras}			no
q_{letras}				sí
$q_{dígitos}$				sí
q_{error}				no

Ahora completa TRANSICIONES en `afd_localidades.py` y habilita las pruebas del paso 3.3.

Checkpoint 3. Conversión y minimización

- ☐ Cada par estado-clase tiene una sola transición.
- ☐ q_{letras} y $q_{dígitos}$ son finales.
- ☐ No se fusionan: una letra adicional es válida desde el primero, pero no desde el segundo.
- ☐ Las pruebas del AFD producen el resultado predicho.

Paso 4. Diseñar antes de programar

4.1 Tabla de reglas léxicas

Completa las expresiones o literales de LARK. Usa la tabla como contrato: si cambias un nombre, deberás cambiar las pruebas y documentar la decisión.

Token	Patrón o literal	Acción	Atributo
LOC	/_____/	emitir y registrar	
NUM	/_____/	emitir	
IF	if"	emitir	
THEN		emitir	
ELSE		emitir	
WHILE		emitir	
DO		emitir	
SKIP		emitir	
TRUE, FALSE		emitir	
ASSIGN		emitir	
LE		emitir	
PLUS, MINUS, TIMES		emitir	
EQ, NOT, AND, OR		emitir	
SEMI, LPAR, RPAR		emitir	
WS	clase importada de LARK	omitir	no aplica

4.2 Resolver una colisión

El lexema `if` satisface también el patrón de `LOC`. El lexer básico no consulta al parser para decidir. En LARK combinaremos una prioridad explícita con un límite léxico:

```
IF.2: /if(?![A-Za-z0-9])/
LOC: /[A-Za-z]+[0-9]*/
```

La prioridad resuelve el empate cuando la entrada es exactamente `if`; la anticipación negativa `(?![A-Za-z0-9])` impide que la palabra reservada corte una localidad más larga. Así, `if2` e `ifX` pueden reconocerse completas como `LOC`.

Predice:

Entrada	Token esperado	Razón
<code>if</code>		
<code>if2</code>		
<code>ifX</code>		

4.3 Especificar pruebas antes del cambio

Escribe el resultado esperado para:

```
X := (Y + 2) <= 10 && true;
```

Compara tu lista con la prueba `test_operadores_y_delimitadores`, pero no retires todavía su decorador `@unittest.skip`.

Checkpoint 4

La tabla debe permitir responder, para cada categoría: qué reconoce, qué emite, qué conserva y con qué prueba se verificará.

Paso 5. Implementar con LARK

5.1 Versión mínima

El archivo inicial contiene:

```
from lark import Lark

GRAMATICA_LEXICA = r"""
LOC: /[A-Za-z]+[0-9]*/
%import common.WS
%ignore WS
"""

lexer = Lark(
    GRAMATICA_LEXICA,
    parser=None,
    lexer="basic",
)

tokens = list(lexer.lex("X"))
```

`parser=None` mantiene el trabajo en la fase léxica; `lexer="basic"` selecciona un lexer independiente; `lex()` recorre la entrada y produce tokens.

5.2 Incorporar numerales

1. Agrega NUM a GRAMATICA_LEXICA.
 2. Ejecuta `python probar_lexer.py ejemplos/minimo.imp`.
 3. Habilita `test_localidades_y_numerales`.
 4. Ejecuta las pruebas y registra el resultado.
- Commit o registro del cambio: _____

5.3 Incorporar palabras reservadas

1. Agrega las ocho palabras reservadas con prioridad superior a LOC y con el límite léxico del modelo de IF.
 2. Habilita `test_palabras_reservadas_no_son_localidades`.
 3. Añade un caso propio con una localidad que empiece igual que una palabra reservada.
- Caso propio y resultado: _____

5.4 Incorporar símbolos

Agrega operadores, asignación, secuenciación y paréntesis. Después:

```
python -m unittest -v pruebas_lexer.py
python probar_lexer.py ejemplos/flujo.imp
python probar_lexer.py ejemplos/control.imp
```

Habilita `test_operadores_y_delimitadores`. No modifiques la salida esperada para hacer pasar una implementación incorrecta; compara primero con la tabla aprobada.

Evidencia 5

Conserva una captura o transcripción legible de:

- la ejecución de las pruebas habilitadas;
- la secuencia producida para `flujo.imp`;
- la secuencia producida para `control.imp`;
- el commit o registro que corresponde a cada incremento.

Paso 6. Atributos y tabla inicial de localidades

6.1 Observar atributos

Cada token de LARK conserva su tipo, lexema y posición. Ejecuta:

```
python probar_lexer.py ejemplos/flujo.imp
```

Selecciona dos tokens del mismo tipo con lexemas distintos:

Tipo	Lexema	Línea	Columna

¿Qué información permanece igual y cuál cambia?

6.2 Construir la tabla

Completa `construir_tabla_localidades` en `lexer_imp.py`. La tabla de esta práctica sólo registra:

- lexema de la localidad;
- línea y columna de la primera aparición;
- número total de apariciones.

No agregues tipos, valores ni alcances: aún no contamos con información semántica para hacerlo.

Habilita `test_tabla_de_localidades` y ejecuta:

```
python probar_lexer.py ejemplos/flujo.imp --tabla
```

Checkpoint 6. Separar responsabilidades

Completa:

- El lexer puede registrar que apareció X en _____.
- Todavía no puede afirmar que X fue _____.
- El tipo de X se comprobará en la fase de _____.

Paso 7. Errores, pruebas y entrega

7.1 Diagnóstico léxico

Ejecuta:

```
python probar_lexer.py ejemplos/error.imp
```

El carácter @ no inicia ningún patrón. El controlador captura `UnexpectedCharacters` e informa línea y columna. Esta práctica se detiene en el primer error: no inventaremos una corrección ni una estrategia de recuperación.

7.2 Activar y ampliar pruebas

Habilita `test_caracter_desconocido`. Después agrega, como mínimo, un caso propio de cada clase:

Clase	Entrada	Resultado esperado
Mínimo correcto		
Varias categorías		
Palabra reservada frente a LOC		
Operador de dos caracteres		
Espacios y saltos de línea		
Carácter desconocido		
Caso frontera propio		

Checkpoint 7. Prueba completa

- ☐ Todas las pruebas habilitadas terminan correctamente.
- ☐ Cada prueba tiene un propósito identificable.
- ☐ Los errores informan una posición relacionada con la entrada.
- ☐ No se usa el parser para resolver decisiones léxicas.
- ☐ No se añadieron comentarios u otras construcciones no aprobadas para IMP.

Criterios de entrega

Criterio	Evidencia observable
Especificación	Tabla de tokens consistente con IMP y con la convención concreta.
Fundamento formal	Expresiones regulares justificadas y AFD de localidades funcional.
Implementación	Lexer independiente con LARK; reglas legibles y nombres consistentes.
Atributos	Tipo, lexema y posición visibles; tabla inicial de localidades correcta.

Pruebas	Casos correctos, colisiones, espacios, operadores compuestos y error léxico.
Documentación	README.md, instalación, uso, decisiones y limitaciones.
Evolución	Historial de GIT que permita reconocer incrementos y verificaciones.
Reporte	Introducción, desarrollo, evidencias, resultados, errores, conclusiones, tiempos y vínculo al repositorio.

Lista final

- ☐ requirements.txt fija la versión de LARK.
- ☐ Los scripts no conservan TODO pendientes de la práctica.
- ☐ Las pruebas ya no están omitidas.
- ☐ Los ejemplos pueden ejecutarse desde las instrucciones del README.md.
- ☐ El reporte distingue lo construido por el equipo de lo automatizado por LARK.
- ☐ La salida léxica queda preparada para ser consumida por la práctica sintáctica.

Bitácora y cierre

Etapa	Estimación	Tiempo real
Preparación y recuperación		
Expresiones regulares y AFD		
Diseño de reglas		
Implementación incremental		
Tabla, pruebas y documentación		
Total		

Reflexión final para el reporte

1. ¿Qué decisión léxica resultó más difícil y con qué evidencia la resolviste?
2. ¿Qué diferencia observaste entre describir un patrón, reconocerlo con un AFD e implementarlo con LARK?
3. ¿Qué información deberá conservarse sin cambios cuando se integre el analizador sintáctico?

Conclusión

El analizador construido materializa la primera transformación interna del compilador: recibe caracteres, reconoce lexemas, emite tokens y conserva atributos verificables. Las expresiones regulares especifican las categorías; los autómatas explican su reconocimiento; y LARK automatiza parte de la ejecución, pero no decide el vocabulario, los nombres, las prioridades ni las pruebas. El contrato documentado permitirá integrar esta salida con el analizador sintáctico en una práctica posterior.

Referencias

- [1] Winskel, G. (1993). *The Formal Semantics of Programming Languages: An Introduction*. The MIT Press.
- [2] Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.^a ed.). Pearson Educación. ISBN 978-970-26-1133-2.
- [3] Thain, D. (2026). *Introduction to Compilers and Language Design* (2.^a ed.). University of Notre Dame.
- [4] Vilar Torres, J. M. (2010). *Estructura de los compiladores e intérpretes*. Universitat Jaume I.
- [5] Vilar Torres, J. M. (2010). *Analizador léxico*. Universitat Jaume I.
- [6] Lark. (s.f.). *API Reference y Grammar Reference*. <https://lark-parser.readthedocs.io/>